Project Documentation

1. Introduction

Project Title: Smart Library Request Workflow in ServiceNow

Team Members:

- KALAIVANI S – Team Lead
- REEMA K M – Team Member
- PANBARASI B – Team Member
- HEMALATHA V – Team Member

2. Project Overview

Purpose:
The Smart Library Request Workflow is designed to streamline library book requests and approvals within ServiceNow. Users can request books, track status updates, and receive notifications, while librarians can manage approvals efficiently. The system also provides dashboards and reports for administrators to monitor book requests and user activity.

Features:

- User login with role-based access (User & Librarian)
- Borrow request submission with validation
- Approval and rejection workflow for librarians
- Email and system notifications for request status updates
- Dashboard displaying total requests, requests by user, and pending approvals
- Monthly and real-time reporting on borrow requests

3. Architecture

Frontend:

- Built with React.js using functional components and hooks
- State management with Redux
- Routing via React Router
- Axios used for API calls to the backend

Backend:

- Node.js with Express.js framework
- RESTful API endpoints for CRUD operations on users, books, and borrow requests
- Middleware for authentication, authorization, and error handling

Database:

- MongoDB collections for Users, Books, and Borrow Requests
- Mongoose used for schema definition and database interaction
- Borrow Requests reference User and Book collections for data integrity

4. Setup Instructions

Prerequisites:

- Node.js (v18+)
- MongoDB (local or Atlas cluster)
- npm or yarn package manager

Installation:

2. **Clone the repository:** https://github.com/kalaisivakumar13-sudo/Smart-Library-Request-Workflow-in-ServiceNow.git

3. **Install dependencies:**
```
cd client
npm install
cd ../server
npm install
```
4. **Configure environment variables in .env file:**
```
PORT=5000
MONGO_URI=[MongoDB URI]
JWT_SECRET=[JWT Secret]
```

5. Folder Structure

Client (React Frontend):

```
client/
├── public/
├── src/
│   ├── components/
│   ├── pages/
│   ├── redux/
│   ├── services/
│   ├── App.js
│   └── index.js
```

Server (Node.js Backend):

```
server/
├── controllers/
├── models/
├── routes/
├── middleware/
├── config/
├── server.js
```

6. Running the Application

Frontend:

```
cd client
npm start
```

Backend:

```
cd server
npm start
```

Frontend runs on port 3000, backend on port 5000.

7. API Documentation

Endpoints:

- POST /api/auth/login – User login
- POST /api/auth/register – User registration
- GET /api/books – Retrieve all books
- POST /api/borrow-requests – Submit a borrow request
- PUT /api/borrow-requests/:id/approve – Approve borrow request (librarian)
- GET /api/reports/monthly – Monthly request summary

Example Response:

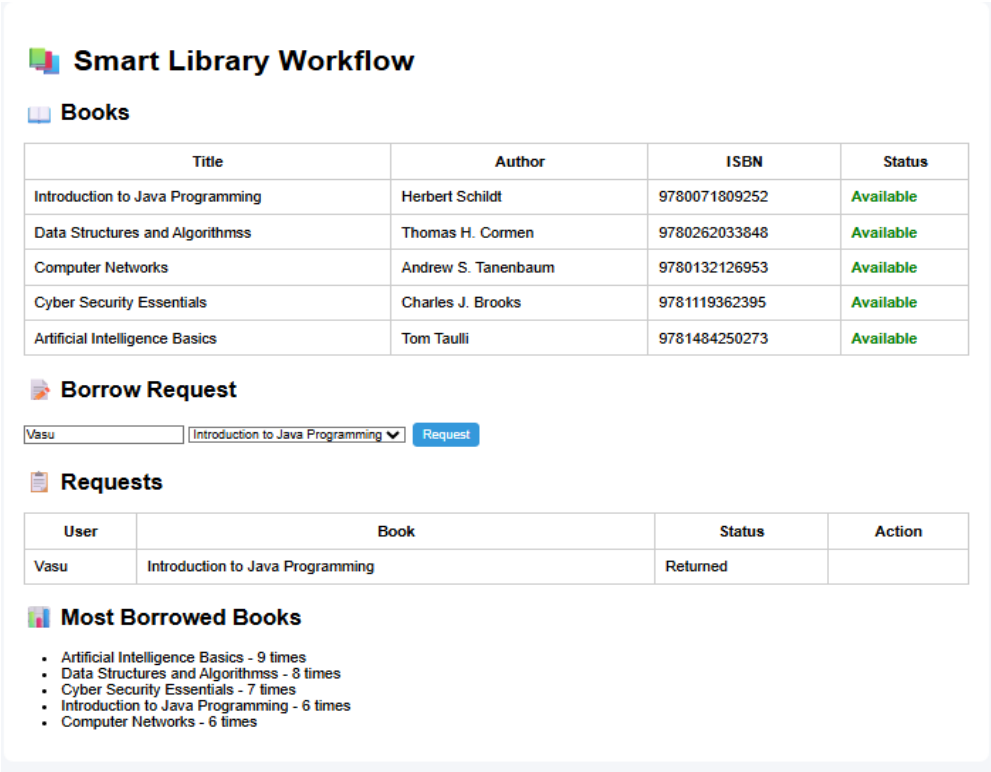
```
{
  "success": true,
  "data": [ ... ]
}
```

8. Authentication

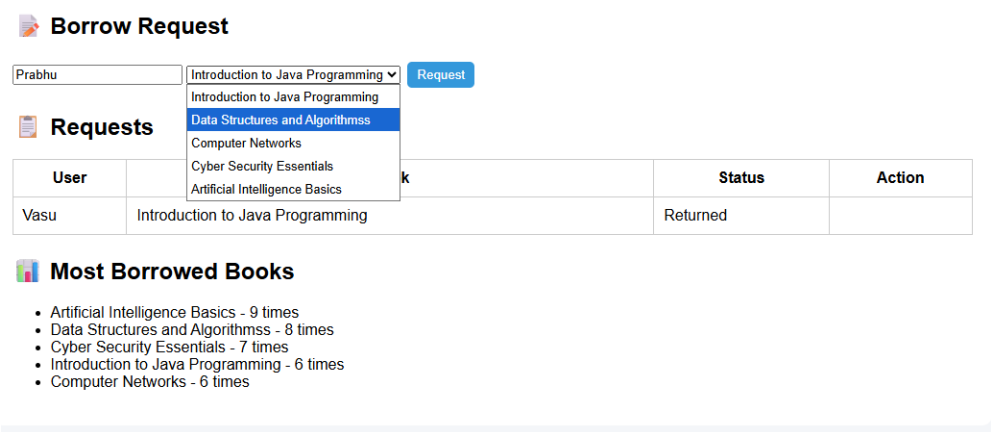
- JWT-based authentication and authorization
- Tokens stored in local storage and sent in headers
- Role-based access control: User vs Librarian
- Middleware checks token validity and user roles for protected routes

9. User Interface

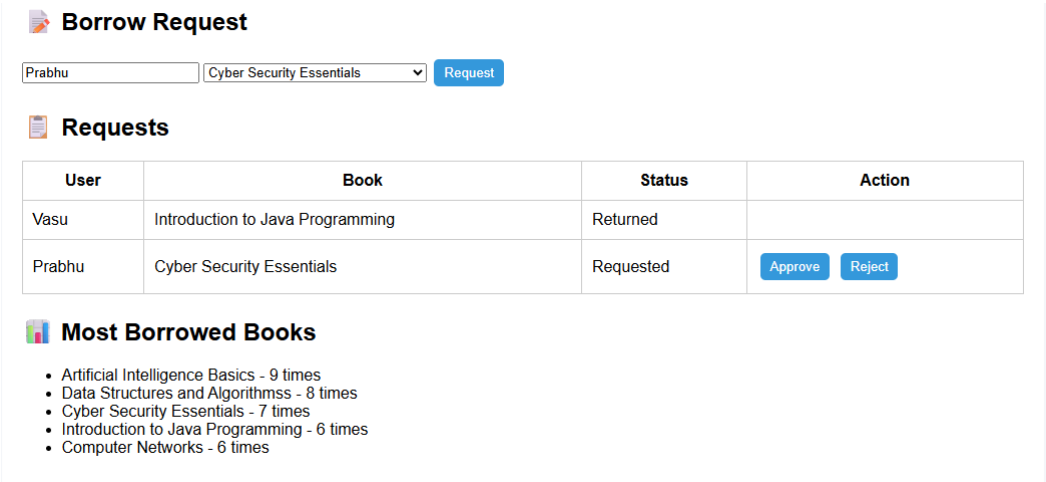
- Screenshots or GIFs showing:
  - Login Page



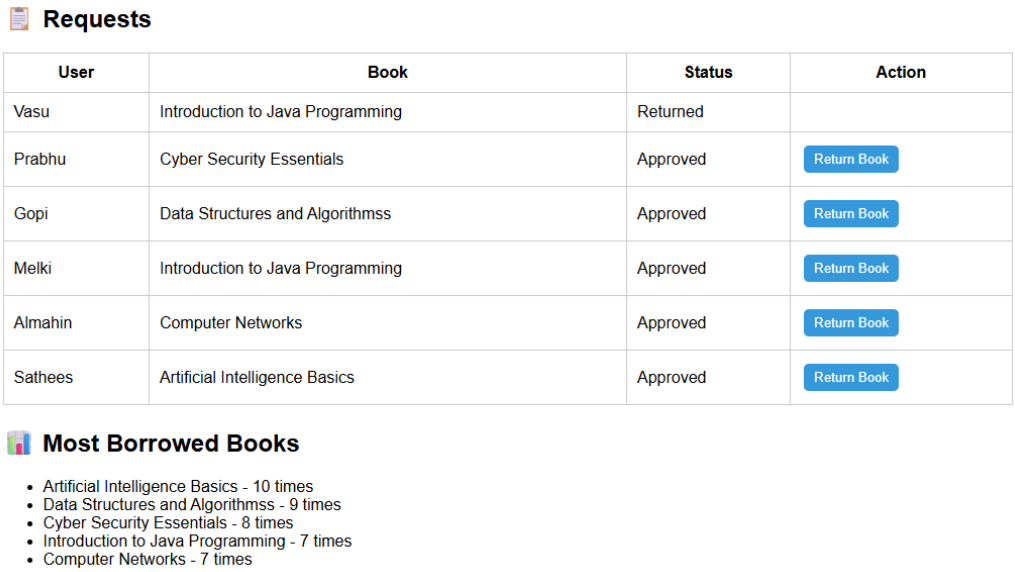
  - Borrow Request Form



- Approval Dashboard



- Reports and Charts



10. Testing

- Backend unit testing using Jest
- API integration testing
- Manual UAT performed for all user workflows

11. Screenshots or Demo

- Demo link: https://drive.google.com/drive/folders/11TfQdluQ-6ZhUIIDX7y4pP9dPvWy18KO?usp=drive_link

12. Known Issues

- Dashboard may experience slight delays with large datasets
- Minor UI inconsistencies on mobile devices

13. Future Enhancements

- Add search and filter functionality for books
- Email notifications for overdue or unreturned books
- Analytics for user activity trends
- Improved mobile responsiveness